

LLM Engineering Assignment - Short Answers (Part A)

Q1. An LLM is an AI model trained on huge amounts of text to understand and generate language. It learns patterns by predicting the next word from examples seen during training.

Q2. LLMs are called large because they use massive datasets and billions of parameters. More data and parameters help them learn complex language patterns.

Q3. Language models evolved from rule-based systems to statistical models, then RNNs, LSTMs, and finally Transformers. Each step improved accuracy, context understanding, and scalability.

Q4. RNNs and LSTMs process text sequentially, making them slow and weak at remembering long contexts. Transformers solve this using self-attention and parallel processing.

Q5. Self-attention lets every word focus on the most relevant words in a sentence. This helps the model understand long-range relationships efficiently.

Q6. Hallucination produces incorrect facts, knowledge cutoff limits recent information, and context windows restrict input length. These can be reduced using reliable data, retrieval, and careful prompting.

Q7. Open-weight models like Llama and Gemma can be downloaded and customized. Closed models like GPT are accessed through APIs and are easier to use but less flexible.

Q8. Computers understand numbers, not raw text. Tokenization converts text into tokens and token IDs that the model can process.

Q9. Character tokenization handles unknown words but creates long sequences. Word tokenization is simple but struggles with new words, while subword tokenization balances both.

Q10. BPE starts with characters and repeatedly merges the most frequent pairs. This creates useful subword tokens over time.

Q11. WordPiece chooses merges that improve prediction quality, while SentencePiece works directly on raw text. SentencePiece is useful for multilingual languages.

Q12. Special tokens mark the beginning, end, padding, or unknown words. They help the model understand sequence structure.

Q13. Attention masks tell the model which tokens are real and which are padding. This prevents padding from affecting predictions.

Q14. Token IDs are only identifiers and have no meaning. Embeddings convert them into meaningful vectors.

Q15. One-hot vectors are sparse and memory intensive, while dense embeddings are compact and capture semantic relationships.

Q16. The embedding layer works like a lookup table that retrieves vectors for token IDs. These vectors start randomly and become meaningful during training.

Q17. Input embeddings convert tokens into vectors before processing, while output embeddings help predict the next token. Weight tying reuses the same matrix to save parameters.

Q18. Positional embeddings tell the model the order of words. Without them, 'Dog bites man' and 'Man bites dog' would appear identical.

Q19. Contextual embeddings change depending on surrounding words. For example, 'bank' has different vectors in financial and river contexts.

Q20. Cosine similarity measures how close two vectors are. A value of 1 means very similar, 0 means unrelated, and -1 means opposite.